Thesis of the end-of-study project

To obtain the State Engineering Diploma

Major: Computer Engineering

STM32Cube Competitor Benchmark

Authored by :

Fares Dkhili

Defended on September 2026, before the jury composed of :

Mrs. Anissa Omrane	FST	- President
Mrs. Sana Younes	FST	- Examiner
Mr. Aymen Zemrani	FST	- Academic supervisor
Mrs. Sirine Eljazi	STMicroelectronics	- Professional supervisor

Class of : 2025/2026

Dedicated to

To my dear parents, whose unconditional love, patience, and countless sacrifices have carried me to this moment. Everything I have achieved is a reflection of the values you taught me.

To my family and my friends, who never stopped believing in me and who lifted me through every doubt and every long night. This work is as much yours as it is mine.

Fares Dkhili.

Acknowledgments

This project marks the end of my engineering studies, and it could not have come together without the people who guided and supported me along the way.

I am deeply grateful to Ms. Sirine Eljazi, my supervisor at STMicroelectronics, for her trust, her availability, and the technical insight she shared throughout this work. Her guidance shaped both the project and the engineer I have become. I also thank the entire STMicroelectronics team for the warm welcome and for making my time within the company a genuine learning experience.

I would like to express my sincere thanks to Mr. Aymen Zemrani, my academic supervisor at the Faculty of Sciences of Tunis (FST), for his rigorous follow-up, his sound advice, and his constant encouragement from the first day to the last.

My thanks also go to the members of the jury: to Ms. Anissa Omrane for the honor of chairing the jury, and to Ms. Sana Younes for accepting to examine and evaluate this work. Their time and attention are sincerely appreciated.

I extend my gratitude to the teaching staff of the Faculty of Sciences of Tunis at the University of Tunis El Manar, whose dedication over these years laid the foundations on which this project was built.

Finally, my heartfelt thanks to my family and friends, whose unwavering support and patience accompanied me through every stage of this journey.

Résumé

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

Mots clés : xxx, xxx, xxx, xxx.

Summary

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

Key Words : xxxx, xxxx, xxx, xxx.

ملخص

ملخص بالعربية، تأكد من أن الترجمة منطقية

الكلمات المفتاحية
مثال

List of Figures

List of Tables

Listings

Contents

Chapter 1

General Description of the Project

Introduction

This chapter introduces the host company and the context of the project. The existing problems are detailed to define the specifications and clarify the requested work. We then present the theoretical foundations relevant to the project and explain the adopted work methodology.

1.1 Host company

This section provides an overview of STMicroelectronics, its history, the ST Tunis site, and the technically oriented marketing and application support team in which this project was carried out.

1.1.1 Presentation

STMicroelectronics is a global semiconductor company dedicated to enhancing people's lives through innovative electronic solutions [1]. With one of the most extensive product portfolios in the industry, ST serves over 200,000 customers worldwide, achieving net revenues of $13.27 billion in 2024 [2].

The company was established in June 1987 as SGS-THOMSON Microelectronics, following the merger of SGS Microelettronica (Italy) and Thomson Semiconductors (France). The name "STMicroelectronics" was officially adopted in May 1998. Figure 1.1 shows the company's official logo.

Figure 1.1: STMicroelectronics logo.

With more than 80 sales and marketing offices and 14 primary manufacturing locations across the globe, the corporation employs over 50,000 people [1].

1.1.2 STMicroelectronics vision and markets

STMicroelectronics has prioritised technological innovation in its strategy for over 25 years. It makes market-driven investments in technology development with the goal of commercialising cutting-edge innovations that create immediate value for its customers [3]. Today, with a robust pipeline, ST delivers products that are smaller, faster, more energy-efficient, and equipped with new features. Driven by a global lean manufacturing culture, ST addresses three main end markets: automotive, industrial, and personal electronics — and offers engagement models ranging from application-specific embedded systems to application-specific standard products.

1.1.3 STMicroelectronics Tunis

The ST Tunis site, located in Technopark El Ghazela, began operations in 2001 with nine engineers. Today it has expanded to more than 200 employees working across design, tools, applications, quality, and support functions [1]. The facility participates in every stage from the initial design of new circuits to the final verification of their functionality before mass production.

The Microcontroller Division (MCD) is dedicated to the design of microcontrollers, demonstration platforms, and solutions for consumer use. It is responsible for the development of drivers and embedded software — specifically the validation and development of libraries and firmware that drive ST microcontrollers. Figure 1.2 shows the organisational structure of the MCD division at the Tunis site; the Maintenance team, highlighted, is where this project was carried out.

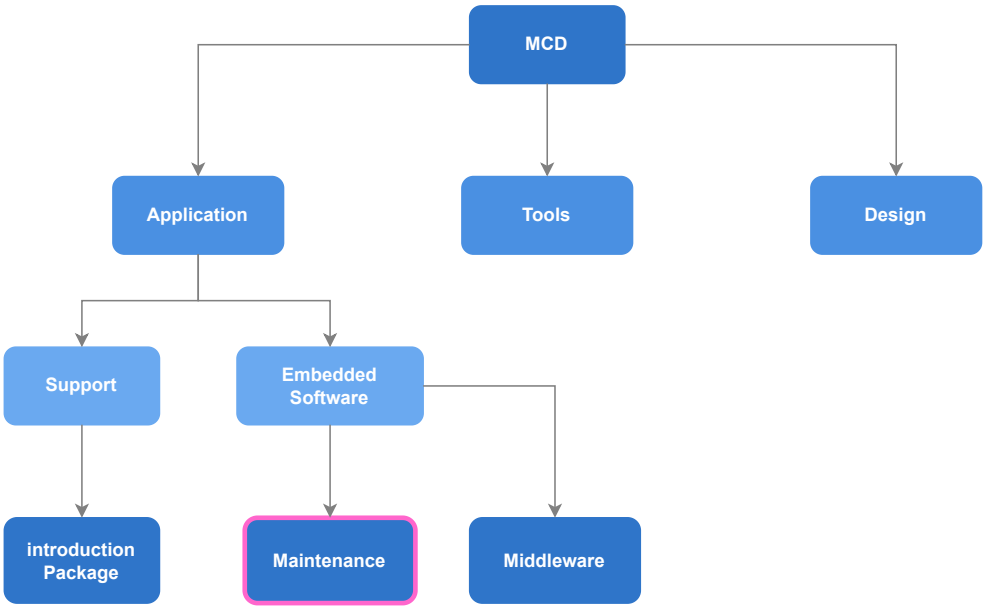

Figure 1.2: Organisational structure of the Microcontroller Division (MCD) at ST Tunis. The Maintenance team (highlighted) is the host team of this project.

This project was carried out in the **Technical Marketing and Application Support** team under MCD. This team includes two sub-teams:

- **The GitHub team**: supports customers via GitHub, publishes updates of HAL drivers, and maintains example projects to facilitate application development based on STM32 microcontrollers.
- **The Maintenance and Validation team**: ensures customer satisfaction through bug maintenance and functional validation of manufactured microcontrollers and their associated HAL versions. This team represents the junction linking software and hardware design processes to implementation and industrialisation.

1.2 Project presentation

1.2.1 Project framework

As part of our training in Computer Engineering at the Faculty of Sciences of Tunis (FST), University of Tunis El Manar (UTM), we conclude our education with an end-of-studies project (PFE). In this context, our project — entitled *STM32Cube Competitor Benchmark* — took place within STMicroelectronics Tunis over a period of six months (February–September 2026).

1.2.2 Project context

The embedded microcontroller market is highly competitive. When an engineer evaluates an MCU for a new design, the choice between STMicroelectronics and competing vendors (GigaDevice GD32, NXP, Texas Instruments, Renesas) often depends on measurable differences: firmware footprint, execution speed, middleware maturity, and development-tool productivity. The technical marketing team at ST needs objective, reproducible data that quantifies where the STM32Cube ecosystem outperforms its competitors, where gaps exist, and what concrete enhancements would close those gaps.

At the start of this project, no systematic benchmarking methodology existed within the team. Cross-vendor comparisons were ad-hoc, performed manually on a case-by-case basis, and difficult to reproduce or communicate convincingly to stakeholders.

1.2.3 Problematic

Producing actionable competitive intelligence for the marketing team requires answering precise questions: for a given software stack (e.g. USB device, FreeRTOS integration), running identical scenarios on ST and on a competitor, how do the two platforms compare in flash and RAM footprint, execution performance, and middleware completeness? And where ST lags, what specific enhancements — at the HAL, middleware, or configuration level — would close the gap?

Answering these questions rigorously raises several challenges:

1. **Scenario design**: each benchmark must exercise the same functionality on both platforms under identical, well-defined conditions so that the comparison is fair and the results are meaningful.

2. **Porting fidelity**: reproducing an STM32 reference scenario on a competitor requires preserving exact runtime semantics — task structure, timing, synchronisation, reporting format — to avoid comparing apples to oranges.

3. **Result extraction and interpretation**: raw numbers (kilobytes, cycles, milliseconds) must be attributed to comparable architectural layers and interpreted in terms of engineering trade-offs, not merely listed.

4. **Scale and reproducibility**: the team needs to cover multiple vendors and multiple stacks with consistent methodology, so that results are comparable across campaigns and over time.

1.2.4 Proposed solution

Our solution is a structured benchmarking methodology centred on three deliverables:

1. **Benchmark scenarios**: for each software stack under evaluation (USB device stack, FreeRTOS, etc.), we design a set of concrete, reproducible scenarios that both the STM32 reference and the competing platform must run identically. Each scenario exercises a specific aspect of the stack (e.g. HID, CDC, and MSC for USB device).

2. **Comparative results and interpretation**: for each scenario, we extract per-layer footprint figures, execution metrics, and qualitative observations, then interpret them to show where ST is stronger, where the competitor leads, and why.

3. **Enhancement proposals**: where the data reveals a gap unfavourable to ST, we identify the root cause and propose targeted improvements — whether at the HAL driver level, in middleware configuration, or in linker/startup overhead.

To support this workflow at scale and ensure reproducibility, we developed two supportive tools in parallel:

- **MCU Workflow Studio** — a cross-vendor firmware footprint analysis application that automates linker-map comparison, layer classification, and result export (detailed in Chapter 2).

- **MCU Benchmark Copilot Agent** — a custom AI agent system that accelerates benchmark creation, porting, and debugging while enforcing fairness and semantic equivalence (detailed in Chapter 3).

These tools are parallel engineering contributions that enhance the benchmarking effort; they are not the benchmarks themselves. The core deliverable remains the benchmark results and the enhancement proposals they yield.

1.2.5 Mission and objectives

The mission of this internship is to provide the STMicroelectronics technical marketing team with rigorous, reproducible, and interpretable competitive benchmark data. Specifically, the project aims to:

1. Design benchmark scenarios for selected software stacks, ensuring fairness and semantic equivalence between ST and each competitor.

2. Execute the benchmarks on representative hardware, extracting flash/RAM footprint, execution performance, and qualitative observations per scenario.

3. Compare and interpret the results layer by layer, highlighting where ST excels and where gaps exist.

4. Formulate concrete enhancement proposals for STM32Cube where the data reveals room for improvement.

5. Develop supportive tools (footprint analyser, benchmark agent) to make the methodology reproducible and scalable beyond this project.

1.3 Theoretical foundations

This section presents the background concepts essential for understanding the technical aspects of the project: the STM32 microcontroller ecosystem, firmware memory footprint, and the linker-map artefact that our tools consume.

1.3.1 STM32 microcontrollers

1.3.1.1 Overview

A microcontroller unit (MCU) is an integrated circuit that combines a processor core, memory (flash and RAM), and peripherals on a single chip. It is designed for embedded applications where cost, power, and real-time determinism are critical constraints.

1.3.1.2 The STM32 family

The STM32 family from STMicroelectronics is a series of 32-bit microcontrollers based on ARM Cortex-M processor cores, produced since 2007 [4]. The family spans four macro groups — high-performance, mainstream, ultra-low-power, and wireless — with more than ten product sub-families. Each sub-family targets a different balance of processing power, memory, peripherals, and energy efficiency.

The ARM Cortex-M profile is optimised for embedded applications requiring low cost and low power consumption [5]. Cortex-M cores range from the M0 (simplest, lowest gate count) to the M7 and M33 (highest performance, with DSP and optional floating-point units). STM32 devices span this entire range.

1.3.1.3 The STM32Cube ecosystem

STMicroelectronics provides a comprehensive software ecosystem around its STM32 hardware [6]:

- **STM32CubeMX**: a graphical configuration tool that generates initialisation code for peripherals, clocks, and middleware.
- **STM32CubeIDE**: an integrated development environment based on Eclipse, with build, flash, and debug support.
- **STM32Cube firmware packages**: per-family packages containing the HAL and low-level (LL) drivers, middleware (FreeRTOS, USB, FatFS, LwIP, mbedTLS), board support packages (BSP), and example projects.
- **STM32CubeProgrammer**: a flashing and option-byte configuration utility.
- **STM32CubeMonitor**: a real-time variable monitoring tool.

It is this ecosystem — not merely the silicon — that we benchmark against competitors. When we compare "ST versus GD32," we compare the full software stack each vendor provides for a given use case, because that is what an engineer evaluates when choosing a platform.

1.3.2 Firmware memory footprint

The memory footprint of firmware is the amount of non-volatile (flash) and volatile (RAM) memory it occupies once compiled and linked. Flash stores read-only code and constant data; RAM stores read-write data (global and static variables, plus stack and heap at run time).

Footprint is a first-class metric in MCU benchmarking for several reasons. Flash and RAM are fixed, on-chip resources; every kilobyte consumed by the vendor's software stack is a kilobyte unavailable for application logic. A smaller footprint enables the use of a cheaper device with less memory, directly affecting bill-of-materials cost. It also leaves headroom for future features and reduces power consumption in flash-retention scenarios.

Comparing footprint across vendors is non-trivial because each vendor structures, names, and layers its code differently. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, `GPIO_PinInit` on NXP, and `R_IOPORT_Open` on Renesas. A fair comparison must map these correspondences and attribute costs to comparable architectural layers (application, middleware, low-level driver, system runtime).

1.3.3 Linker maps

The linker map is a text file produced by the linker at the end of the build process. For the IAR Embedded Workbench for ARM (EWARM) toolchain [15], this file lists every module and function that was linked into the final image, together with its code size (read-only), constant-data size (read-only), and variable-data size (read-write). It also records the memory sections, placement addresses, and total resource usage.

The linker map is the single input to our footprint analysis tool. By parsing it, we can reconstruct the complete static memory budget of a firmware build without instrumenting or executing the code. This makes the measurement perfectly reproducible: the same source code, compiler version, and optimisation level always produce the same map.

1.4 Work methodology

We adopted a "divide and conquer" approach [7], decomposing the project into well-scoped tasks with clear deliverables and deadlines:

1. **Task 1 — Literature and ecosystem study**: research the STM32Cube ecosystem, competing vendor ecosystems (GD32, NXP, TI, Renesas), linker-map formats, and existing benchmarking practices.

2. **Task 2 — Benchmark scenario design**: define the software stacks to benchmark, select representative boards for ST and each competitor, and design fair, reproducible scenarios with identical observable behaviour on both sides.

3. **Task 3 — Benchmark execution and porting**: build and validate each scenario on the STM32 reference, port it to the competitor platform preserving semantic equivalence, compile both under IAR EWARM, and extract results.

4. **Task 4 — Result comparison and interpretation**: compare per-layer footprint, execution metrics, and qualitative observations; identify where ST excels and where gaps exist; formulate enhancement proposals.

5. **Task 5 — Supportive tool development** (parallel): design and implement MCU Workflow Studio (footprint analysis) and the MCU Benchmark Copilot Agent (benchmark creation and porting assistant) to make the methodology scalable and reproducible.

6. **Task 6 — Documentation and report**: document the methodology, tools, and results in this report.

Conclusion

In this chapter, we introduced STMicroelectronics and its Tunis site, presented the competitive context that motivates cross-vendor benchmarking, and defined the core mission: design benchmark scenarios, execute them on ST and competitor platforms, compare and interpret the results for the marketing team, and propose targeted enhancements where ST can improve. Two supportive tools developed in parallel make this methodology reproducible and scalable. The theoretical foundations — the STM32Cube ecosystem, firmware footprint, and linker maps — provide the vocabulary for the chapters that follow. Chapter 2 and Chapter 3 present the supportive tools, while subsequent chapters present the benchmark results themselves.

Chapter 2

A Cross-Vendor Firmware Footprint Analysis Tool

Introduction

Comparing STMicroelectronics against a competing vendor is, at bottom, a measurement problem: the same workload is built for both platforms and the resulting binaries are weighed against one another. One of the most revealing measurements is the *memory footprint* of the firmware — how much flash and RAM a given software stack consumes once compiled and linked. Footprint is easy to define but tedious to compare fairly across ecosystems, because two vendors rarely name, split, or organise their code the same way. To make this comparison reproducible rather than artisanal, we designed and built a dedicated desktop and command-line application, *MCU Workflow Studio*, that ingests the linker output of two builds and produces a layer-aware, explainable footprint comparison between an STM32 reference and a competitor.

This chapter documents that tool as an engineering contribution in its own right. We first state the problem it solves and the requirements we set for it, then describe its architecture and its end-to-end pipeline, the implementation choices behind its deterministic core and its optional language-model assistance, how it plugs into the wider benchmarking effort, and finally its concrete value and its current limitations.

2.1 The cross-vendor footprint problem

The flash and RAM budget of a microcontroller unit (MCU) is a hard constraint. Every kilobyte of code competes with application logic for a fixed amount of on-chip memory, and it ripples into device cost, power, and the headroom left for future features. When we benchmark a software stack — a Universal Serial Bus (USB) device stack, a real-time operating system integration, and so on — the footprint each vendor charges for the same functionality is therefore a first-class metric.

Measuring it fairly is the difficult part. A modern build links application code against a vendor software development kit (SDK) whose drivers, middleware, and startup code are structured to that vendor's own conventions. The same peripheral initialisation appears as `HAL_GPIO_Init` on STM32, as `GPIO_PinInit` on one competitor, and as `fsl_gpio` routines on another; equivalent middleware sits in different modules, and the boundary between "driver" and "system runtime" is drawn differently in each toolchain. Doing the comparison by hand means reading two linker maps side by side, guessing which symbol on one side corresponds to which on the other, and summing kilobytes into comparable buckets — slow, subjective, and impossible to reproduce exactly two weeks later.

We set the tool a small number of firm requirements. It had to be **deterministic and reproducible**, so that the same inputs always yield the same verdict. It had to be **layer-aware**, grouping symbols into architectural layers so that "ST spends more in the hardware abstraction layer (HAL)" can be stated precisely. Its symbol matching had to be **explainable**, attaching a confidence and a reason to every correspondence rather than hiding behind a black box. It had to treat **ST as the reference** against which each competitor is measured. And it had to **export** its results in formats the rest of the report can consume directly.

2.2 Design and architecture

The tool is organised as a thin set of user interfaces wrapped around a deterministic comparison core, with an SQLite-backed knowledge base alongside and an optional, off-by-default block of language-model assistance. Figure 2.1 shows the arrangement. Data flows top to bottom: a build's linker map enters through the ingestion layer, is classified into architectural layers, is compared symbol by symbol in the core, and leaves as a set of exported artefacts.

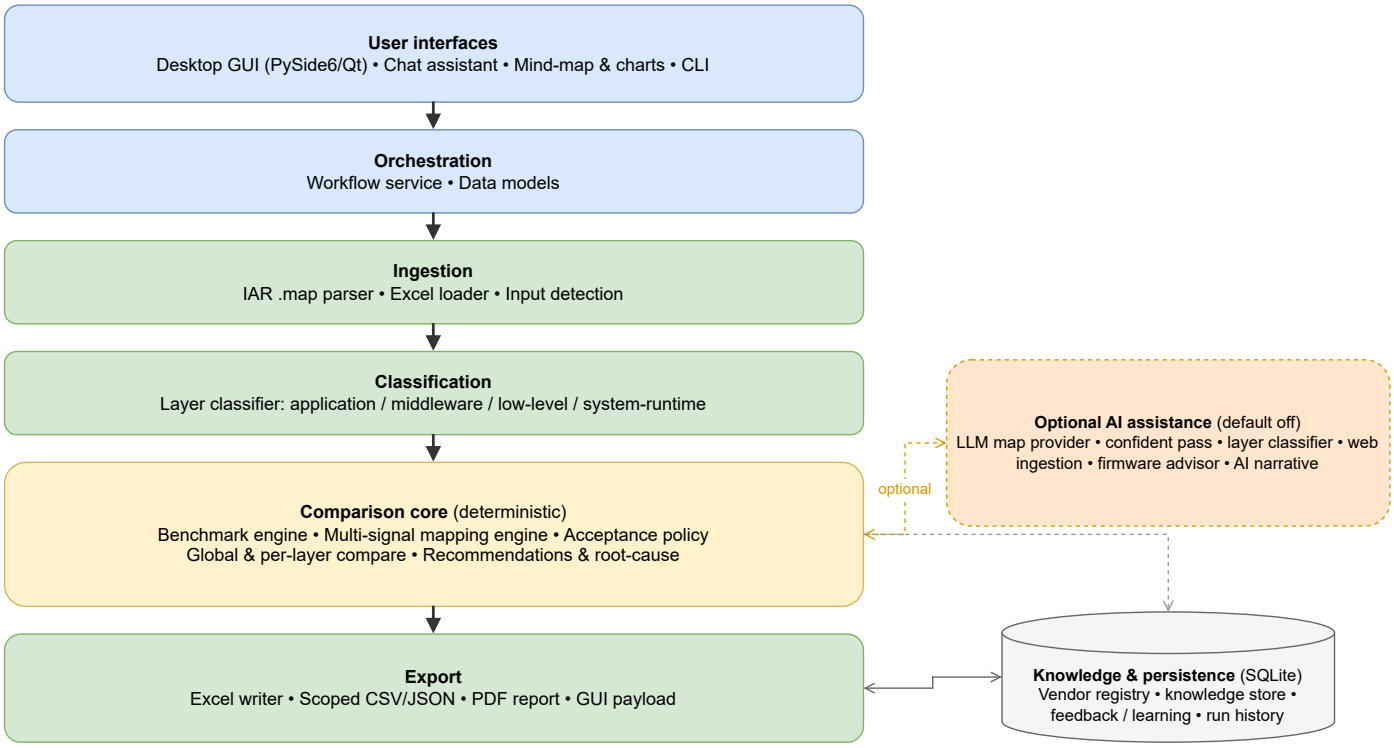

Figure 2.1: Layered architecture of the footprint-analysis tool. A deterministic comparison core (ingestion, classification, mapping, comparison, export) is wrapped by the user interfaces and backed by an SQLite knowledge base; an optional, off-by-default AI-assistance column refines the mapping without ever sitting on the critical path.

Two interfaces sit on top of the same engine: a desktop graphical user interface (GUI) built with PySide6 (the Qt binding for Python) [9], which adds interactive charts and a project-scoped chat assistant, and a command-line interface for scripted and reproducible runs. Both call into a single *workflow service* that owns the data models and drives every stage. Persistence is handled by SQLite [13]: a vendor registry of aliases and naming prefixes, a knowledge store of confirmed correspondences, an optional feedback store, and a history of past runs. The block of language-model features on the right is genuinely optional — the deterministic path produces a complete result without it — and we return to it in Section 2.4.3. Table 2.1 lists what the tool consumes and what it produces.

Table 2.1: Principal inputs and outputs of the tool.

Inputs	**Description**
IAR EWARM `.map` (×2)	Linker maps of the source (STM32) and target (competitor) builds; the only required inputs.
Prior workbook (`.xlsx`)	Optional earlier results re-used as a baseline.
Vendor identifiers	Source and target vendor names (default `stm32`/`nxp`) that select the alias and prefix tables.
Outputs	**Description**
`benchmark_results.xlsx`	Per-layer and global flash/RAM comparison with a verdict.
Scoped CSV/JSON/XLSX	Filtered exports for downstream processing.
Review queue (JSON/CSV)	Low-confidence mappings prioritised for manual confirmation.
Mapping diagnostics (JSON)	Per-pair signal breakdown and acceptance decisions.
GUI payload (JSON)	Pre-rendered data for the desktop interface.
PDF report (optional)	Eight-page narrative summary with charts.

2.3 The benchmarking pipeline

A benchmark run is a fixed sequence of stages, sketched in Figure 2.2. The solid path is purely deterministic; the dashed stages are the optional language-model steps, which are disabled by default and can be switched on without changing anything else. After validating its inputs, the tool parses both linker maps into a common in-memory model — build metadata, an overall footprint, and the list of modules and functions with their code and data sizes. Each symbol is then classified into an architectural layer, and the vendor knowledge (aliases, prefixes, prior matches) is prepared.

The heart of the run is the mapping stage, where each STM32 symbol is matched to its closest competitor counterpart by combining several similarity signals into a single, explainable score. The accepted matches are then aggregated: the tool computes the flash and RAM totals per layer and globally for both sides, compares them, and emits a verdict together with recommendations and a root-cause breakdown of where the difference comes from. Finally it persists the quality of the pair and composes its exports; the optional advisor, PDF report, and run-history entry are produced only if requested.

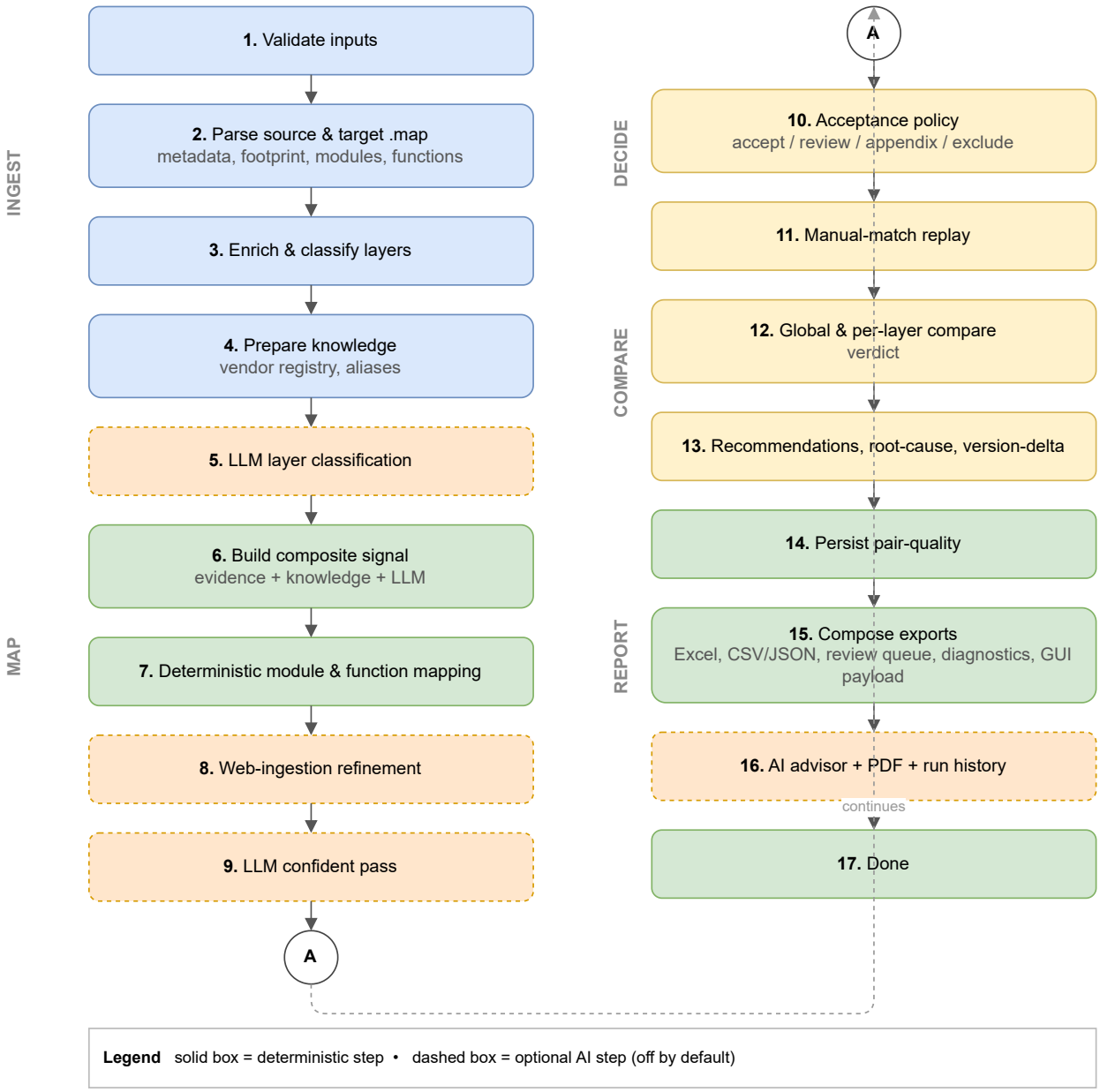

Figure 2.2: End-to-end benchmark pipeline, from linker-map ingestion to report generation. Dashed stages are optional AI-assisted steps that are disabled by default; the solid deterministic path runs end to end on its own.

A second, lighter mode produces a single-map footprint summary — validate, parse, classify, and write a footprint workbook — which is useful when only one build is available or to sanity-check a map before a full comparison.

2.4 Implementation

The tool is written entirely in Python (≥ 3.11) [8], with a small, deliberately conventional dependency set summarised in Table 2.2. We favoured the standard library wherever it was sufficient: all persistence uses

the built-in SQLite engine [13], and every network call to a language model goes through the standard `urllib` module rather than a vendor SDK, which keeps the executable small and the dependency surface auditable. The whole application can be packaged into a stand-alone Windows executable with PyInstaller [14].

Table 2.2: Implementation stack and the role of each external dependency.

Concern	**Technology**	**Role in the tool**
Language	Python $\geq$ 3.11 [8]	Entire code base.
Desktop GUI	PySide6 / Qt [9]	Windows, charts, chat assistant.
Tabular export	pandas [10]	Scoped CSV/JSON/Excel export.
Excel I/O	openpyxl [11]	Reads prior workbooks, writes results.
Charts & PDF	matplotlib [12]	Figures embedded in the PDF report.
Persistence	SQLite [13]	Knowledge base, run history, caches.
Packaging	PyInstaller [14]	Stand-alone Windows executable.

2.4.1 Parsing linker maps and classifying layers

The single required input is the linker map emitted by the IAR Embedded Workbench for ARM (EWARM) toolchain [15]. The parser reads this map and reconstructs, for each module and function, the read-only code and data it contributes and the read-write data it reserves. From these we derive the two footprint figures the benchmark cares about: flash occupancy as the sum of read-only code and read-only data, and RAM occupancy as the read-write data. This is a *static* measurement taken from the linked image; it captures what the firmware costs to store, not its dynamic stack or heap behaviour at run time, and we are careful to present it as such.

Each symbol is then assigned to one of four architectural layers — application, middleware, low-level driver, or system runtime — by a deterministic classifier driven by module names, known vendor prefixes, and section attributes. This layering is what lets the final report say not merely that one firmware is larger, but *where* it is larger, which is far more useful when the goal is to understand a vendor's engineering trade-offs. Figure 2.3 details the decision flow.

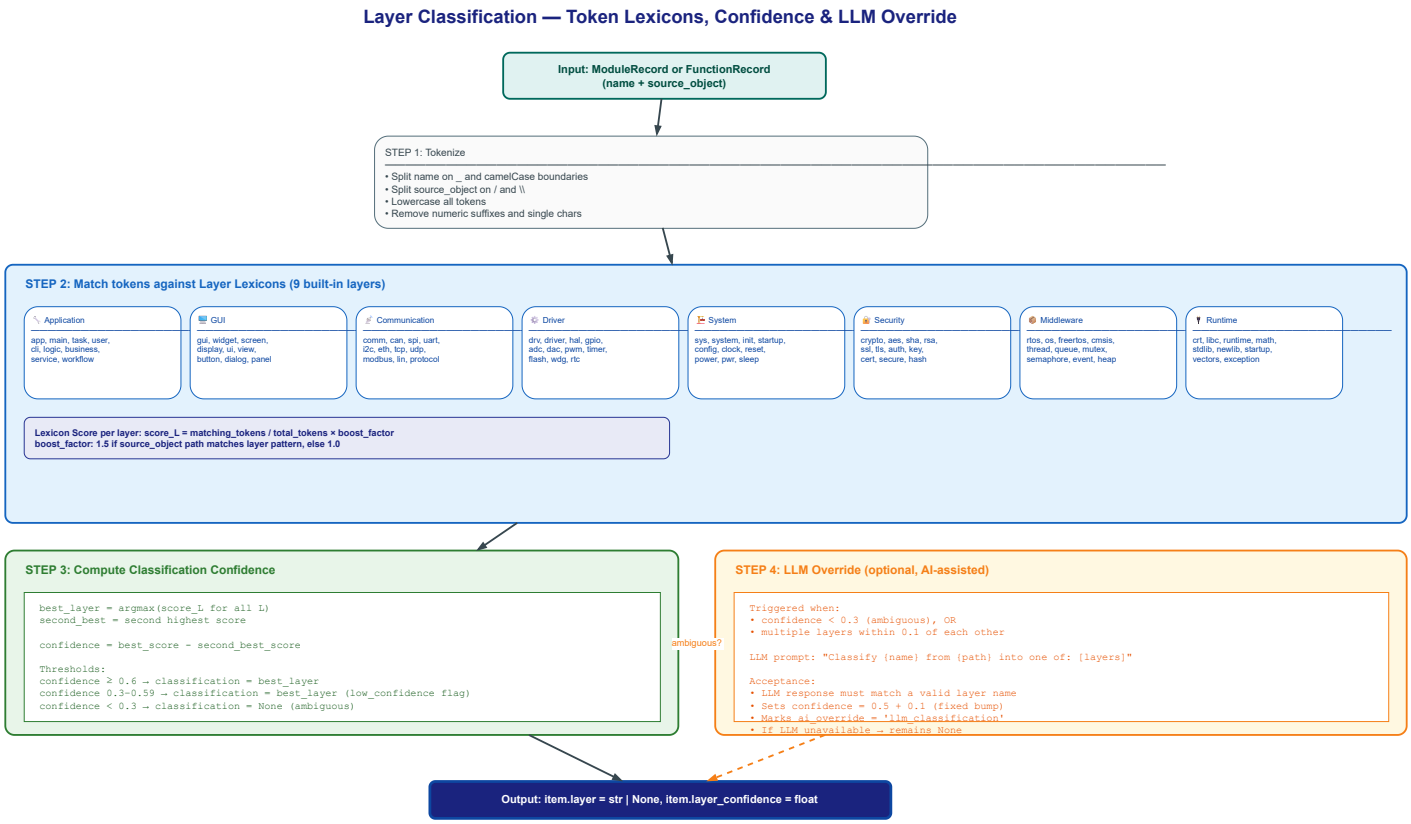

Figure 2.3: Layer-classification decision flow. Each symbol's tokens are matched against priority-ordered lexicons (system, middleware, low-level, application); the first match determines the layer with an associated confidence.

2.4.2 The multi-signal mapping engine

Matching ST symbols to their competitor counterparts is the part that, done manually, costs the most time and is the least repeatable, so it received the most care. Rather than rely on name similarity alone — which fails precisely when two vendors name the same function differently — the engine scores each candidate pair by combining a dozen weighted signals into a single confidence value. Name and role similarity carry the most weight, followed by footprint proximity, shared external references, and call-signature, object-type, and operation cues; aliases from the vendor registry, surrounding context, and historical priors contribute the rest. Figure 2.4 shows the relative weights, and Figure 2.5 expands the computation and penalty mechanics of each signal.

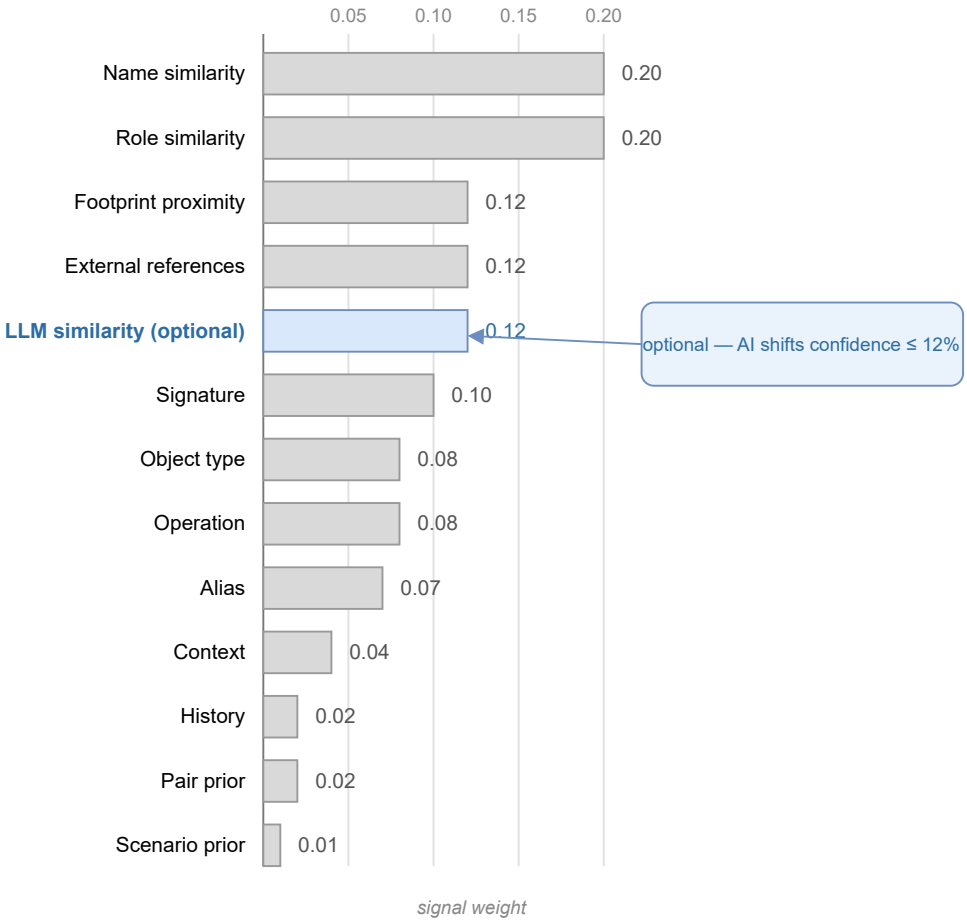

Figure 2.4: Relative weights of the signals combined by the deterministic mapping scorer. Name and role similarity dominate; the optional LLM-similarity signal (highlighted) is capped so that AI assistance can shift a candidate's confidence by at most twelve percent.

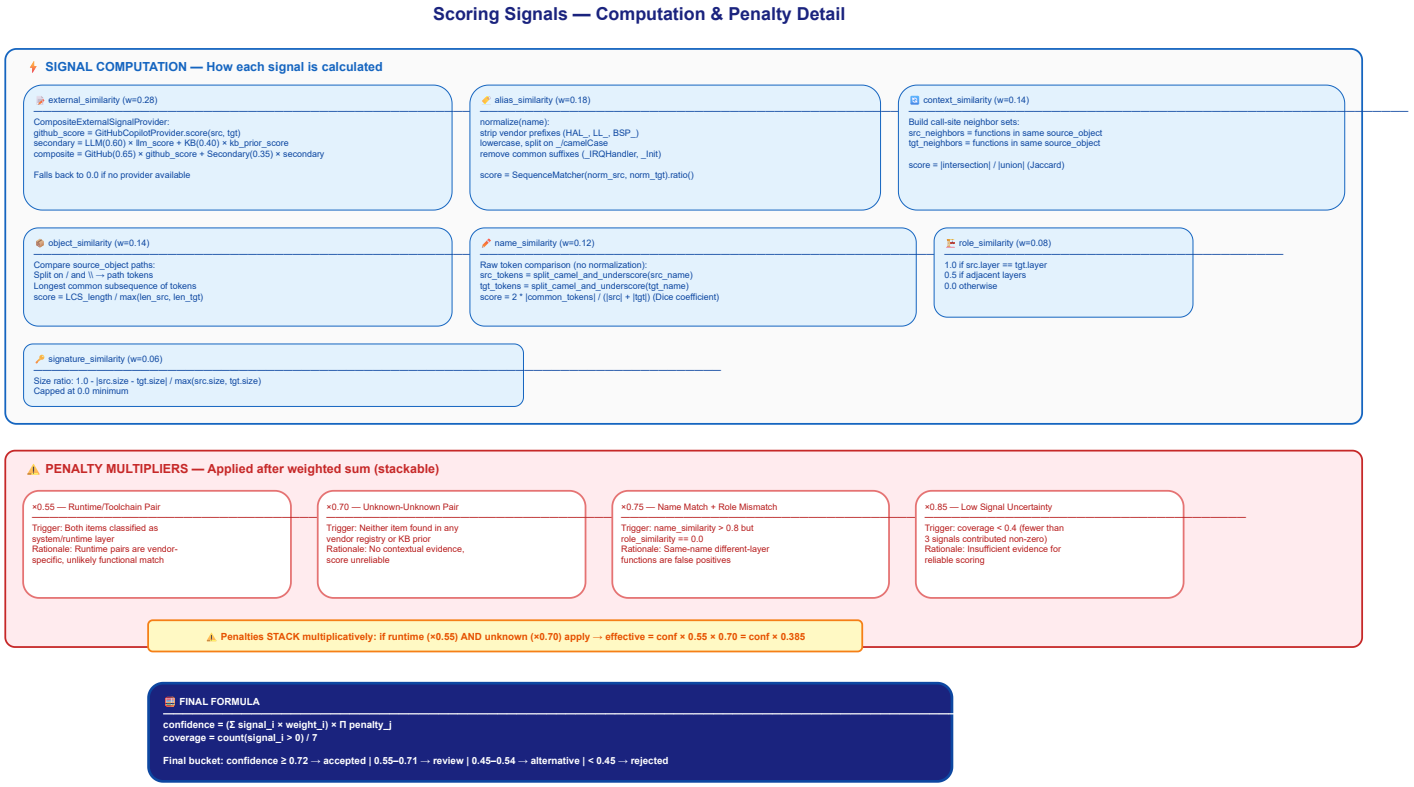

Figure 2.5: Detailed signal computation and penalty system. Seven evidence signals are computed for each candidate pair; penalties for size mismatch and layer conflict reduce the composite score before the acceptance threshold is applied.

The resulting confidence drives a transparent acceptance policy. A correspondence is accepted automatically once its confidence reaches 0.72, queued for manual review above 0.55, kept as a weaker alternative above 0.45, and an unmatched symbol is retained on its own only above 0.80. The engine also records the cardinality of each match — one-to-one, one-to-many, or many-to-one — which matters because vendors routinely split or merge functionality differently, and a one-to-many match is itself a finding about how the two stacks are structured. Every decision, with its per-signal breakdown, is written to the mapping diagnostics so that a reviewer can see exactly why two symbols were paired. Figure 2.6 illustrates the complete flow from candidate generation through pruning, scoring, and threshold-based bucketing.

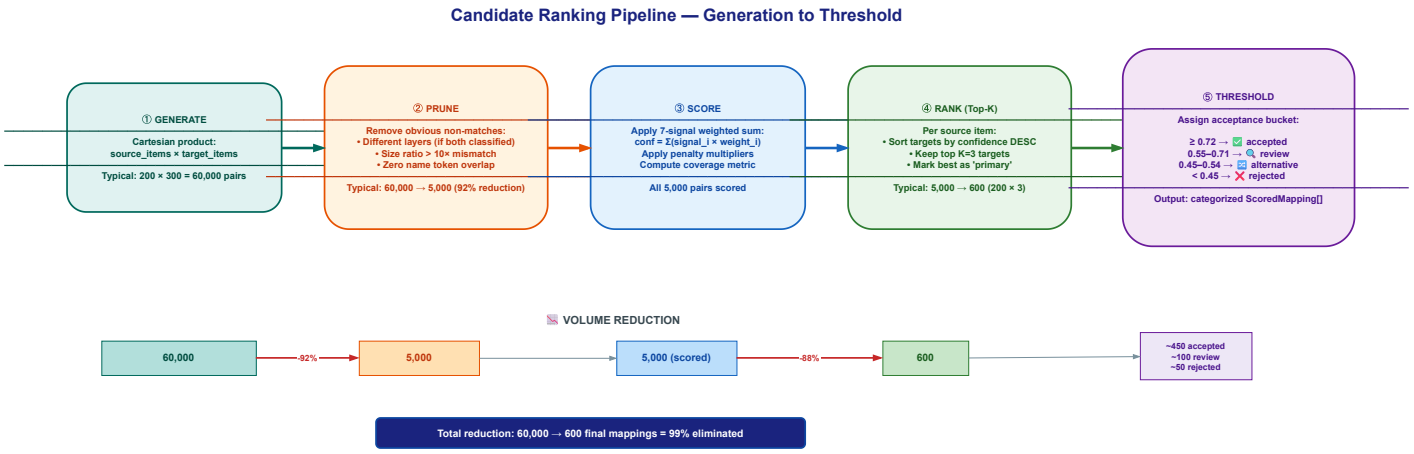

Figure 2.6: Candidate generation, ranking, and threshold flow. The Cartesian product of source and target items is pruned by fast rejection filters, scored on all signals, and bucketed into accepted, review, alternative, and unmatched categories by confidence thresholds.

2.4.3 Optional AI assistance

The tool is best described as a deterministic core with *optional* language-model assistance, not as an "AI tool" in the sense of one that cannot work without a model. Every language-model feature is opt-in and disabled by default, and the engine produces a complete, reproducible result with all of them switched off. When they are enabled, they act in three bounded ways. First, a large language model (LLM) can supply one additional similarity signal during mapping; as Figure 2.4 makes explicit, that signal is weighted and capped so it can move a candidate's confidence by at most twelve percent, and the run stays within a fixed call budget. Second, a "confident pass" may confirm or reject borderline matches, and an advisor may suggest where STM32 firmware could be tightened. Third, the desktop interface offers a chat assistant for exploring a comparison in natural language.

These features can be served by several interchangeable back-ends, listed in Table 2.3, selected automatically in a fixed order or pinned explicitly. They range from a hosted GitHub Copilot endpoint [16] to a fully offline Ollama server [19], so the assistance can run without any data leaving the machine. If a back-end is unavailable or returns something unusable, the tool degrades gracefully to its deterministic output rather than failing. Figure 2.7 shows the provider abstraction and the auto-detection fallback chain.

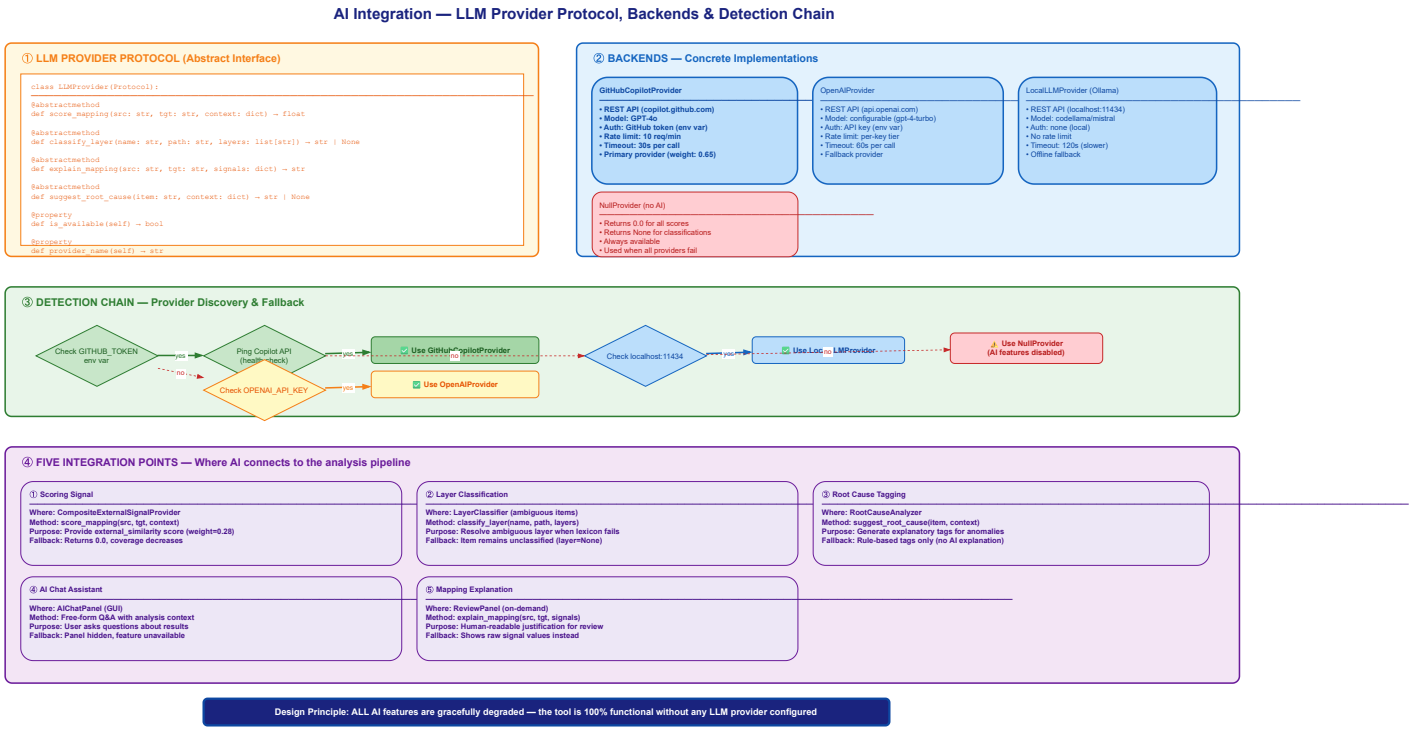

Figure 2.7: AI integration flow. A common protocol abstracts all back-ends; an auto-detection chain probes them in priority order (Copilot, OpenAI, Ollama) and falls back gracefully to purely deterministic operation if none is available.

Table 2.3: Optional language-model back-ends. All are disabled by default; the tool runs fully without them.

Back-end	**Default model**	**Selection / requirement**
GitHub Copilot [16]	Claude (Anthropic) [17]	OAuth device flow; first in the auto-detect order.
OpenAI [18]	GPT-4o	`OPENAI_API_KEY` environment variable.
Ollama [19]	Llama 3.1 (8B)	Local server; fully offline.
GitHub Models [20]	gpt-4.1-mini	Default for the narrative assistant.

2.4.4 Persistence and learning

All state lives in local SQLite databases: the vendor registry, the store of confirmed correspondences, a cache of assistant responses, and a history of runs. A manual confirmation made during review can be replayed on later runs, so the tool improves as it is used on a given vendor pair. We treat this learning, together with the web-ingestion and run-history features, as useful but experimental, and none of it is on the critical path of a comparison.

2.5 Integration into the benchmark workflow

The tool occupies a precise place in our method. Once a software stack has been built for an STM32 board and for the competing vendor's board under identical scenarios, each build's IAR linker map is fed to the tool as the source and the target. Its per-layer and global footprint figures, its verdict, and its root-cause breakdown are exactly the numbers the per-vendor benchmarking chapters of this report quote when they weigh ST's flash and RAM cost against a competitor's for a given stack. In other words, the footprint comparisons reported later are not assembled by hand: they are produced, with an auditable trail, by the tool described here, which is why we document it before presenting any results that depend on it.

2.6 Usage and outputs

A comparison can be driven either from the desktop interface or, for reproducible and scriptable runs, from the command line. A typical invocation names the two maps and their vendors and points at an output directory, as in Listing 2.1.

```
mcu-workflow-cli benchmark \
  --source stm32usbhidclassproject.map --source-vendor stm32 \
  --target nxpusbhidclassproject.map   --target-vendor nxp \
  --out results/
```

Listing 2.1: Command-line invocation of a footprint benchmark.

The run leaves behind the artefacts of Table 2.1: a results workbook with the per-layer and global comparison, scoped CSV/JSON exports, a prioritised review queue of the matches that warrant a human look, the full mapping diagnostics, and — on request — an eight-page PDF report with an executive summary, source-versus-target charts, an architecture-and-module breakdown, a gap analysis, and recommendations. The samples we used to develop and validate the tool are themselves a representative comparison: a USB human interface device (HID) class project built for an STM32U575 board against the equivalent project built for an NXP MCXN947 board [21], both compiled with the same IAR toolchain so that only the vendor software stacks differ.

2.7 Engineering value and limitations

The value of the tool is that it turns a slow, subjective, once-off comparison into a fast, explainable, and repeatable one. A footprint benchmark that previously took an afternoon of careful map-reading now runs in seconds and produces the same answer every time, with a documented reason behind every symbol correspondence and a clear attribution of where each kilobyte of difference originates. That reproducibility is what lets the benchmarking chapters of this report rest their footprint claims on evidence rather than on judgement.

We are equally clear about what the tool does not do. It reads IAR EWARM linker maps; builds produced by other toolchains are out of scope for now, and broadening the parser is the most obvious avenue for future work. Its footprint figures are static, derived from the linked image rather than from execution. The migration-effort estimates and letter grades that the optional PDF report can produce are heuristic indications, not measured quantities, and we label them accordingly wherever they appear. Finally, the language-model features, while useful, are optional and still maturing; the trustworthy core of the tool is, and is meant to remain, fully deterministic.

Conclusion

We designed and built MCU Workflow Studio to remove the manual bottleneck in cross-vendor footprint benchmarking and to make its results reproducible and explainable. Its deterministic core parses two IAR linker maps, classifies every symbol into an architectural layer, maps STM32 symbols to their competitor counterparts with an auditable confidence, and reports per-layer and global flash and RAM differences with a clear root cause — with optional, strictly bounded language-model assistance available on top. The chapters that follow rely on this tool for the footprint figures they report, which is why we have presented it first, as a contribution in its own right.

Chapter 3

An AI-Assisted Agent for Benchmark Creation, Porting, and Debugging

Introduction

The previous chapter presented a tool that analyses the *output* of a benchmark — two linker maps produced by completed builds. However, producing those builds in the first place is itself a substantial engineering task. Each new cross-vendor comparison requires creating a benchmark project for the competitor platform, porting the STM32 reference code while preserving its exact runtime semantics, configuring the IAR Embedded Workbench for ARM (EWARM) project correctly, and debugging the inevitable compiler, linker, and runtime issues that arise when targeting unfamiliar silicon. Done manually, this process is slow, error-prone, and difficult to keep consistent across many vendor comparisons.

To address this, we designed and built a custom GitHub Copilot agent system — the *MCU Benchmark Copilot Agent* — that operates inside Visual Studio Code (VS Code) and automates the reasoning-intensive parts of benchmark creation, porting, building, and debugging [16, 22]. Where MCU Workflow Studio analyses what *exists*, the agent system helps *create* what does not yet exist, and it does so under strict rules that enforce semantic equivalence and benchmark fairness at every step. The two tools are complementary: the agent produces the builds, and the studio analyses their footprint.

This chapter documents the agent as an engineering contribution. We describe the problem it addresses, its architecture and the specialist roles it defines, the reusable skill workflows it exposes, the always-on rules that govern its behaviour, how it integrates with the rest of the benchmarking effort, and its value and limitations.

3.1 The benchmark engineering problem

A single STM32-versus-competitor benchmark involves several non-trivial steps. The STM32 reference project must first be understood in full — task structure, timing, peripheral usage, reporting format, and observable behaviour. A corresponding project must then be created for the target platform, mapping every STM32 hardware abstraction layer (HAL) call to the competitor's equivalent application programming interface (API) without altering the benchmark's semantics. The resulting project must be configured for the IAR toolchain with the correct device selection, startup file, linker configuration, include paths, and compiler defines. Once it compiles, runtime issues — wrong interrupt vectors, misconfigured clocks, broken universal asynchronous receiver-transmitter (UART) output — must be diagnosed from real evidence rather than guesswork. And throughout, the comparison must remain fair: same optimisation level, same measurement scope, same observable output format.

Doing this for one vendor pair and one software stack is manageable. Repeating it for multiple vendors (GD32, NXP, Renesas, Texas Instruments) across multiple stacks (USB device, FreeRTOS, CoreMark) quickly becomes the dominant time cost of the benchmarking campaign. Each port also carries a risk of *semantic drift* — subtle changes in task priorities, timing, or synchronisation that invalidate the comparison without being immediately visible.

We needed an assistant that could enforce the methodology consistently across every port, refuse to guess when evidence was missing, and make its reasoning transparent. A general-purpose large language model (LLM) cannot do this out of the box: it lacks the domain-specific checklists, the structured routing, and the policy of refusing to claim success without hardware evidence. A custom agent system, built on VS Code's Copilot extensibility framework, gave us the ability to encode our benchmarking methodology directly into the tool's behaviour.

3.2 Architecture

The system is structured as a set of cooperating agents and reusable skills, orchestrated by a single router, and governed by a shared set of always-on rules. All configuration lives in a `.github/` directory within the benchmark workspace and is expressed as Markdown files with YAML frontmatter — no compiled code, no external server, no deployment step. The agents run inside VS Code's GitHub Copilot Chat extension [16] and have access to the workspace file system, terminal, and search capabilities.

Figure 3.1 illustrates the layered arrangement. A user request enters the router agent, which classifies it and delegates to the appropriate specialist agent or invokes a skill workflow directly. Specialist agents

operate in isolated context and return a structured result. Skills are stateless checklists and workflows that any agent can invoke.

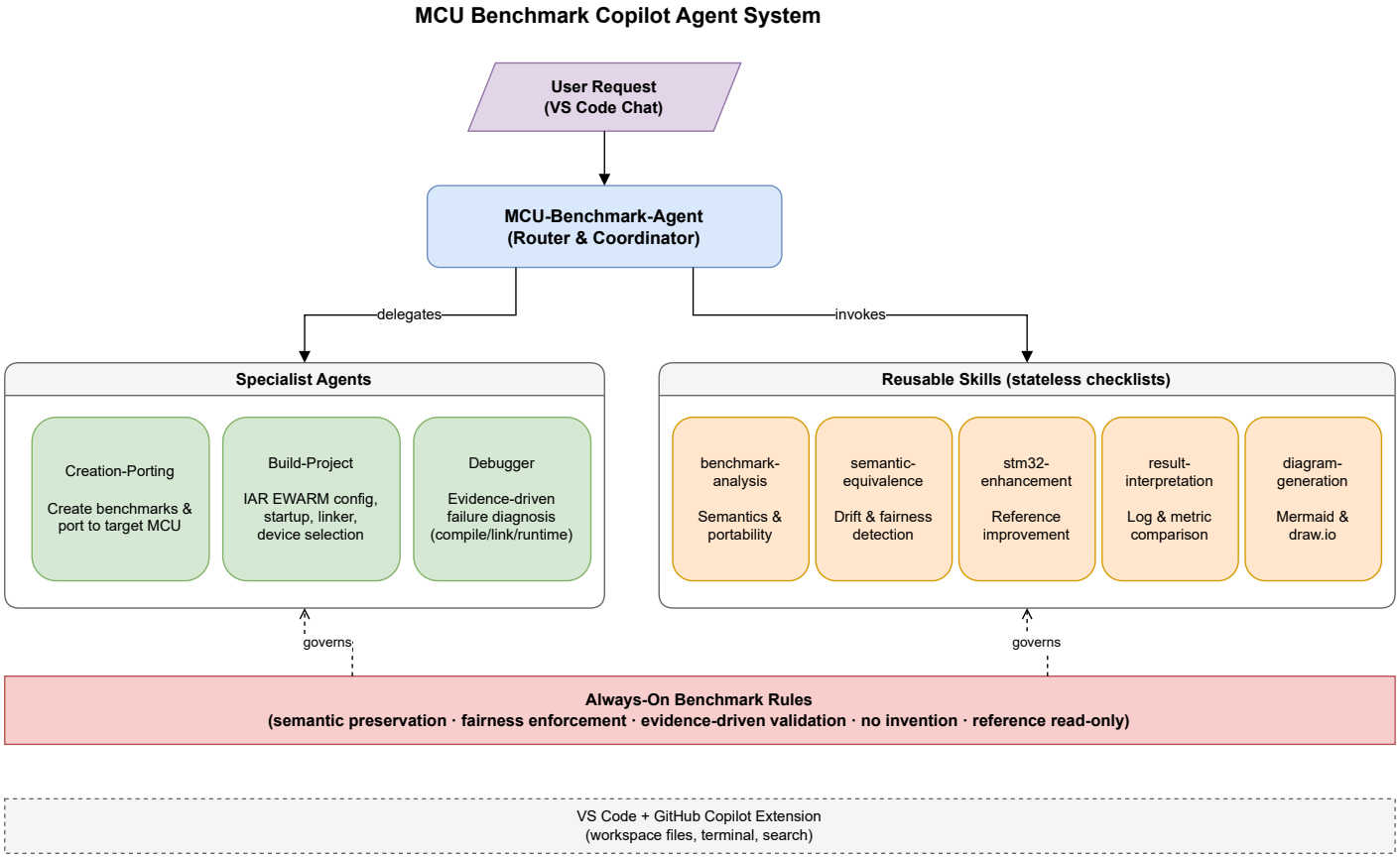

Figure 3.1: Architecture of the MCU Benchmark Copilot Agent system. The router classifies requests and delegates to specialist agents or invokes skill workflows; all operate under the shared always-on benchmark rules within the VS Code environment.

3.2.1 The four specialist agents

Each agent has a defined responsibility, a restricted tool set, and a structured output contract. Table 3.1 summarises them.

Table 3.1: Specialist agents and their responsibilities.

Agent	**Role**	**Responsibility**
MCU-Benchmark-Agent	Router & coordinator	Classifies user requests via a decision tree and delegates to the correct specialist or skill. Has full tool access.
Creation-Porting	Project generation	Creates new benchmark projects and ports STM32 references to target MCU platforms while preserving exact semantics.
Build-Project	Toolchain configuration	Configures IAR EWARM projects: source groups, include paths, compiler defines, startup files, linker files, and device/debugger selection.
Debugger	Failure diagnosis	Diagnoses compiler, linker, and runtime failures from real evidence (build output, UART logs, debugger captures). Refuses to guess.

The router agent's decision tree is deterministic: a request involving project understanding goes to the *benchmark-analysis* skill; a request for cross-vendor porting goes to *Creation-Porting*; build configuration issues go to *Build-Project*; and failures accompanied by real evidence go to *Debugger*. This routing prevents a general-purpose model from attempting tasks outside its configured expertise and ensures that each specialist receives only the context it needs.

3.2.2 The five reusable skills

Skills encode repeatable workflows as structured Markdown checklists. Unlike agents, they do not receive isolated context or delegated reasoning — they provide structure, not autonomy. Table 3.2 lists them.

Table 3.2: Reusable skills and their purposes.

Skill	Purpose	Primary users
benchmark-analysis	File classification, semantics extraction, and portability split of an existing project.	Router, Creation-Porting
semantic-equivalence	Reference-versus-target behaviour comparison and drift detection.	Router, Creation-Porting
stm32-enhancement	Gap-driven recommendations for improving STM32 reference portability.	Router
result-interpretation	Comparative interpretation of benchmark logs, measurements, and result files.	Router, Debugger
diagram-generation	Mermaid and draw.io workflow, architecture, and presentation diagrams.	Router

3.3 Always-on benchmark rules

All agents and skills inherit a shared policy layer — the *always-on benchmark rules* — that constrains the system's behaviour regardless of which specialist is active. These rules encode the core principles of our benchmarking methodology:

- **Reference is read-only.** The STM32 reference project is never modified unless the user explicitly requests it; new target projects are generated instead.
- **Semantic preservation.** A seventeen-point checklist governs what must be identical between reference and target: task count and creation order, priorities, delays, periodicity, synchronisation primitives, suspend/resume and yield behaviour, counters, state transitions, measurement scopes, reporting format, and error behaviour.
- **No invention.** The system does not invent vendor APIs, pin mappings, peripheral choices, or board details. Uncertain hardware details are marked as `TODO` rather than guessed.
- **Evidence-driven validation.** No claim of runtime success is made without actual compiler output, runtime logs, UART captures, or debugger observations.
- **Fairness enforcement.** A fairness checklist — covering optimisation level, clock frequency,

memory configuration, RTOS wrapper differences, measurement instrumentation overhead, and environmental differences — is applied to every comparison and any risk is flagged explicitly.

These rules solve a specific problem with using general-purpose LLMs for embedded work: without them, a model may silently simplify a benchmark, invent an API that does not exist, or claim that code works when it has never been compiled. The always-on policy prevents all three failure modes.

3.4 Semantic preservation in detail

The semantic-equivalence skill deserves elaboration because it is the mechanism that keeps cross-vendor comparisons valid. When the Creation-Porting agent produces a target project, the semantic-equivalence skill is invoked to compare the target against the reference on every point of the checklist. The result is classified into one of five categories:

1. **Equivalent** — behaviour matches within the benchmark definition.
2. **Equivalent with caveats** — behaviour matches, but environment or implementation details affect fairness (e.g. a different tick source).
3. **Drift** — behaviour differs and may affect results.
4. **Approval required** — an intentional deviation that the user must accept.
5. **Unknown** — evidence is insufficient to determine equivalence.

Any result other than category 1 halts the workflow and produces a clear report of what differs and why. This prevents semantic drift from silently propagating into published benchmark numbers.

3.5 Integration with the benchmarking workflow

The agent system occupies a precise position in the benchmarking pipeline. It sits *before* MCU Workflow Studio in the workflow: the agent creates and validates the target project, ensures it compiles under IAR EWARM, and confirms that its runtime behaviour matches the reference. Only then does the build's linker map become an input to MCU Workflow Studio for footprint analysis. The relationship is strictly sequential and complementary — the agent is responsible for *producing* correct builds, and the studio is responsible for *measuring* them. Figure 3.2 shows the end-to-end lifecycle.

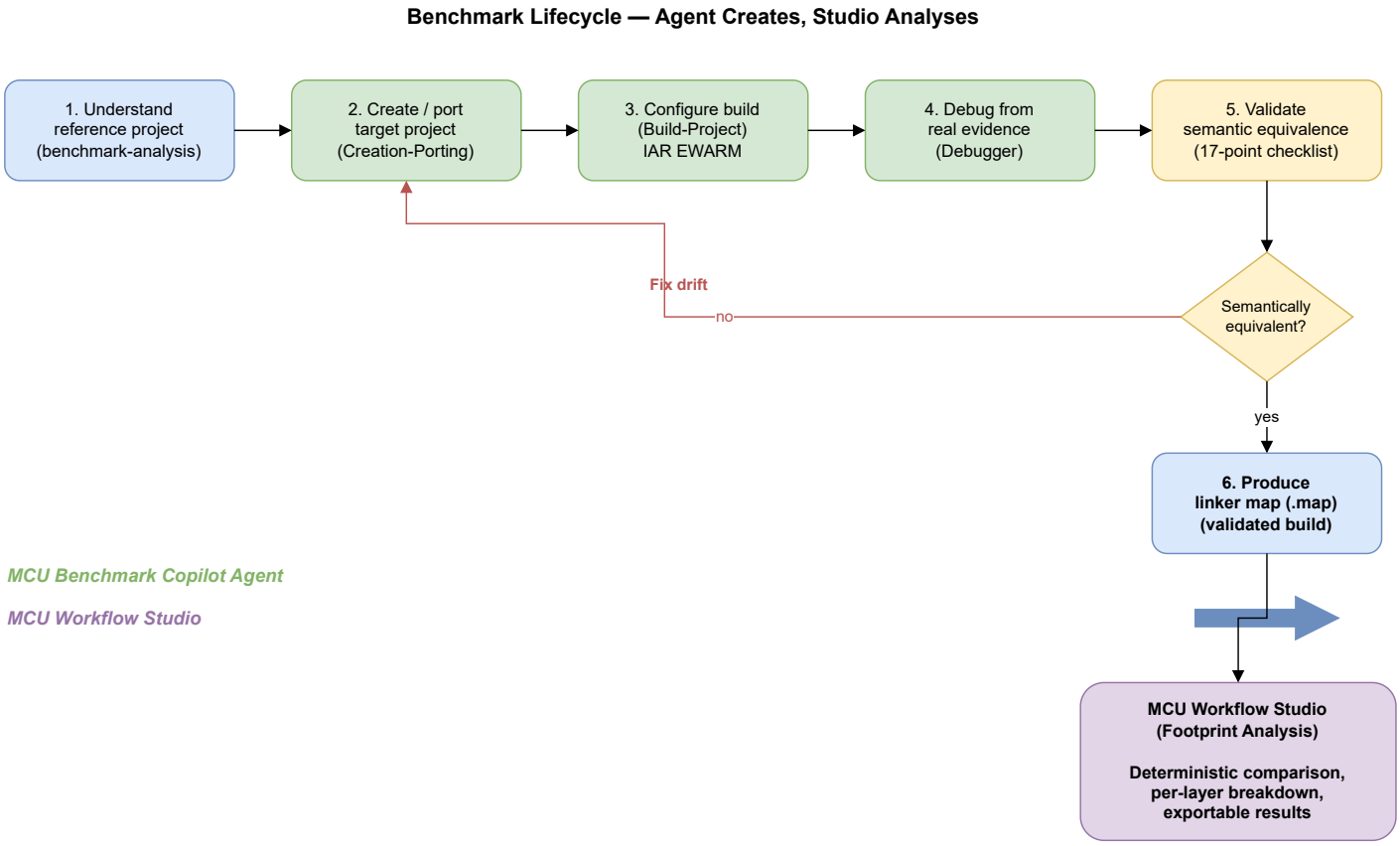

Figure 3.2: End-to-end benchmark lifecycle. The MCU Benchmark Copilot Agent handles steps 1 through 5 (understand, create, build, debug, validate); once semantic equivalence is confirmed, the validated linker map is handed to MCU Workflow Studio for footprint analysis.

This separation keeps each tool focused. The agent does not attempt footprint analysis, and the studio does not attempt code generation or debugging. Together, they cover the full lifecycle of a cross-vendor benchmark: create, port, build, debug, validate, then analyse.

3.6 Implementation

The entire agent system is implemented as a set of Markdown configuration files within a `.github/` directory, leveraging the VS Code GitHub Copilot extensibility framework [16, 23]. No compiled code is required beyond what VS Code and the Copilot extension already provide. The configuration files use YAML frontmatter to declare agent metadata (name, description, tool permissions) and Markdown bodies for the behavioural instructions. Table 3.3 summarises the file structure.

Table 3.3: File structure of the agent system.

Path	Role
`.github/copilot-instructions.md`	Always-on rules inherited by every agent and skill.
`.github/agents/*.agent.md`	One file per specialist agent (4 files).
`.github/skills/*/SKILL.md`	One directory per skill, each containing a structured checklist (5 directories).

The implementation requires no external server, no API keys beyond the user's existing GitHub Copilot subscription, and no deployment pipeline. Adding a new agent or skill is a matter of creating a Markdown file and restarting the Copilot Chat session. This minimal footprint was deliberate: the system should be as easy to maintain and version-control as the benchmark source code itself.

3.7 Engineering value and limitations

The agent system's primary value is *methodology enforcement*. By encoding the semantic-equivalence checklist, the fairness checklist, and the evidence-driven validation policy directly into the tool's behaviour, we ensure that every benchmark port follows the same discipline regardless of which vendor is targeted or how many comparisons are run in parallel. The structured routing prevents the model from operating outside its configured expertise, and the refusal to guess without evidence prevents false claims of success.

In practical terms, the agent reduces the time to produce a correct, validated target project from several hours of manual porting and debugging to a guided session of tens of minutes, with the methodology applied automatically rather than relying on the engineer's memory. The structured output contracts — each agent returns a defined set of sections (assumptions, semantic checklist, fairness risks, validation points) — also make the work auditable and reproducible.

We acknowledge several limitations. The system depends on the quality of the underlying language model; while the always-on rules constrain its behaviour, they cannot prevent all model errors, particularly for vendor APIs that are poorly represented in the model's training data. The tool supports IAR EWARM as the target toolchain; extending it to other compilers (GCC, Keil) would require additional Build-Project agent configurations. Finally, the evidence-driven validation policy means the system cannot fully automate the workflow — human intervention is still required to provide compiler output, flash the board, and capture runtime logs. This is by design: in embedded benchmarking, the hardware is the ground truth, and no amount of static reasoning can substitute for it.

Conclusion

We designed and built the MCU Benchmark Copilot Agent to fill the engineering gap between defining a benchmark and having a validated, compilable target project ready for footprint analysis. Its four specialist agents cover the full creation-to-debugging lifecycle, its five reusable skills encode our methodology as structured checklists, and its always-on rules enforce semantic equivalence, benchmark fairness, and evidence-driven validation at every step. Together with MCU Workflow Studio, it forms a complete toolchain for reproducible, explainable cross-vendor firmware benchmarking: the agent produces correct builds, and the studio measures them.

3.7.0.0.1 Paragraph a

3.7.0.0.2 Paragraph b

Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

Chapter 4

Benchmarking ST against GD32

Introduction

GigaDevice Semiconductor is a Chinese fabless semiconductor company that produces the GD32 series of 32-bit microcontrollers [24]. The GD32 family is often positioned as a pin-compatible, drop-in alternative to STM32 parts, sharing the same Arm Cortex-M cores, similar peripheral sets, and comparable pricing. This close correspondence makes GD32 a particularly relevant competitor: engineers evaluating an MCU platform frequently shortlist both vendors, and the choice often comes down to measurable differences in software-stack quality rather than silicon specifications.

In this chapter we compare the STM32Cube ecosystem against GigaDevice's GD32 firmware libraries on two middleware stacks: the **USB device stack** (exercised through HID and CDC scenarios) and the **FreeRTOS** real-time operating system integration (exercised through basic, cooperative, and preemptive scheduling scenarios). For each stack we present the scenario design, per-scenario footprint and execution-time results, a per-layer ROM breakdown, and an interpretation of the trade-offs the data reveals.

All measurements follow the methodology described in Chapter 1: identical scenarios on both platforms, same compiler optimisation level, same clock frequency, and results extracted from IAR EWARM linker maps using the footprint analysis tool presented in Chapter 2.

4.1 Hardware platforms

Every scenario in this chapter runs on two boards hosting comparable Cortex-M33 parts: the STMicroelectronics **NUCLEO-U575ZI-Q** (STM32U575ZIT6Q) and GigaDevice's **GD32E507** evaluation board (GD32E507ZET6)

Both devices integrate an Arm Cortex-M33 core clocked at 160 MHz for these tests, ensuring that performance differences reflect software-stack efficiency rather than raw silicon speed. Table 4.1 contrasts their key characteristics.

Table 4.1: Board characteristics: ST reference vs GigaDevice GD32.

Characteristic	**ST (STM32U575ZIT6Q)**	**GD32 (GD32E507ZET6)**
Core	Arm Cortex-M33	Arm Cortex-M33
Clock (test)	160 MHz	160 MHz
Flash	2 MB	512 KB
SRAM	786 KB	128 KB
USB	Full-Speed device	Full-Speed device
Security	TrustZone, AES, PKA, HASH	—
Toolchain / IDE	STM32CubeIDE / IAR EWARM	IAR EWARM

The STM32U575 belongs to the ultra-low-power STM32U5 series and offers substantially more flash and RAM, hardware security accelerators, and a richer peripheral set [25]. The GD32E507 targets a similar performance point at a competitive price but with fewer on-chip resources [24]. For our benchmarks, the relevant constant is that *both devices run the same core at the same frequency*, so footprint and execution-time differences are attributable to the vendor's software stack and HAL implementation.

4.2 USB device stack

The Universal Serial Bus (USB) device stack is a critical middleware component for any MCU that needs to communicate with a host PC. We benchmark ST's USB Device Library (part of the STM32Cube middleware) against GigaDevice's USB library, both operating in **Full-Speed** mode and running bare-metal (no RTOS).

4.2.1 Scenario design

We defined two scenarios that cover the most commonly used USB device classes:

1. **HID (Human Interface Device)** — a low-rate interrupt-transfer scenario in which the MCU acts as a USB mouse, sending periodic HID reports to the host. This exercises the enumeration

path, descriptor handling, and interrupt-endpoint transfer logic.

2. **CDC (Communication Device Class)** — a higher-throughput bulk-transfer scenario in which the MCU exposes a virtual COM port. The host sends data to the device, which echoes it back. This exercises bulk IN/OUT transfers, line-coding negotiation, and buffering.

Each scenario was implemented identically on both platforms: same USB descriptors, same endpoint configuration (Full-Speed), same clock source (internal oscillator — HSI48 on STM32, IRC48M on GD32), and same core frequency of 160 MHz. Table 4.2 summarises the scenarios and the metrics collected.

Table 4.2: USB device stack benchmark scenarios.

Scenario	What it exercises	Metrics
HID	Interrupt transfers, enumeration, descriptor handling	ROM/RAM (bytes), init/enum/TX time (ms)
CDC	Bulk transfers, virtual COM port, echo loopback	ROM/RAM (bytes), init/enum/TX/RX time (ms)

4.2.2 Results

4.2.2.1 HID scenario

Table 4.3 presents the total ROM and RAM footprint for the HID scenario.

Table 4.3: USB HID device footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM (total)	16 269	9 175	+77 %
RAM (total)	3 446	9 608	−65 %

The STM32 firmware consumes 77 % more ROM than GD32, but uses 65 % *less* RAM. The RAM advantage is substantial: the GD32 implementation requires 0x2000 bytes (8 KB) of stack and 0x2000 bytes of heap, whereas the STM32 configuration needs only 0x400 bytes (1 KB) of stack and 0x200 bytes (512 bytes) of heap.

Table 4.4 breaks down the ROM consumption by architectural layer.

Table 4.4: USB HID ROM breakdown by layer (bytes).

Layer	**STM32U575**	**GD32E507**	**Difference**
Application	2 580	1 942	+33 %
HAL drivers	10 426	820	+1 171 %
MW stack	2 859	5 743	−50 %

The HAL layer dominates the difference. On STM32, the HAL RCC module alone accounts for 4 560 bytes (44 % of total HAL ROM) and the combined HAL/LL USB driver adds another 4 988 bytes (48 %). By contrast, GD32's HAL layer is minimal at 820 bytes because GigaDevice folds much of the peripheral-initialisation logic into the middleware itself. This architectural choice gives GD32 a smaller HAL at the cost of a larger middleware layer: the GD32 USB library ROM (5 743 bytes) is twice the size of ST's (2 859 bytes).

The STM32 application layer is 33 % larger because the STM32Cube USB library externalises device configuration into user files (`usbd_desc.c`, `usbd_conf.c`), accounting for 985 bytes (38 % of application ROM). On GD32, these configuration concerns are handled inside the middleware. This design gives STM32 users more configuration flexibility at the cost of a larger application footprint.

Table 4.5 shows the execution-time measurements.

Table 4.5: USB HID execution time (ms).

Operation	**STM32U575**	**GD32E507**	**Difference**
Initialisation	9.990	26.084	−61.7 %
Enumeration	213	216	−1.4 %
Data transmission	0.0093	0.0094	−0.7 %

The STM32 USB initialisation is 61.7 % faster than GD32's — a significant difference that reflects more efficient clock-tree configuration and peripheral bring-up in the STM32Cube HAL. Enumeration and data-transmission times are nearly identical, as expected: these phases are dominated by USB protocol timing rather than software overhead.

4.2.2.2 CDC scenario

Table 4.6 presents the total footprint for the CDC scenario.

Table 4.6: USB CDC device footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM (total)	17 049	9 085	+88 %
RAM (total)	4 089	9 683	−57.7 %

The pattern mirrors HID: STM32 uses substantially more ROM (+88 %) but far less RAM (−57.7 %). The CDC scenario adds transmit and receive buffer management, which increases both vendors' footprints relative to HID, but the ratio remains consistent.

Table 4.7 provides the per-layer ROM breakdown.

Table 4.7: USB CDC ROM breakdown by layer (bytes).

Layer	STM32U575	GD32E507	Difference
Application	2 683	1 882	+42.5 %
HAL drivers	10 446	820	+1 174 %
MW stack	2 423	5 709	−57.5 %

The root cause is the same as in HID. The STM32 HAL RCC module (4 560 bytes) and the HAL/LL USB driver (5 006 bytes) together account for 92 % of the HAL ROM. The STM32 application layer is 42.5 % larger due to the externalised USB device configuration files (`usb_device.c`, `usbd_cdc_if.c`, `usbd_desc.c`, `usbd_conf.c`), contributing 1 164 bytes (43 % of application ROM). In return, the STM32 USB middleware library itself is 57.5 % smaller than GD32's, because GigaDevice bundles configuration and driver logic into its middleware layer.

Table 4.8 shows the execution-time measurements.

Table 4.8: USB CDC execution time (ms).

Operation	STM32U575	GD32E507	Difference
Initialisation	9.994	26.076	−61.7 %
Enumeration	218.2	218.5	−0.1 %
Data transmission	0.00988	0.01003	−1.5 %
Data reception	0.00986	0.00993	−0.7 %

Once again, initialisation is where STM32 dominates: 61.7 % faster than GD32. The remaining

operations (enumeration, transmission, reception) are within 1.5 % of each other, confirming that steady-state USB throughput is protocol-bound rather than software-bound.

4.2.3 Interpretation

The USB benchmark reveals a clear architectural trade-off between the two vendors.

4.2.3.0.1 ROM footprint. STM32 consumes 77–88 % more total ROM than GD32 across both scenarios. The dominant contributor is the HAL layer, which is roughly twelve times larger on STM32. Two modules drive this cost: the **HAL RCC** clock-configuration driver (≈4 560 bytes) and the **HAL/LL USB** peripheral driver (≈5 000 bytes). GigaDevice achieves a much smaller HAL by embedding peripheral-initialisation logic inside the middleware library rather than exposing it as a separate, reusable driver layer.

4.2.3.0.2 RAM footprint. The situation is reversed: STM32 uses 57–65 % less RAM. The GD32 implementation defaults to much larger stack and heap allocations (8 KB each vs ≤1.5 KB for STM32). This suggests that the GD32 USB library relies more heavily on dynamic allocation or deep call chains, whereas the STM32Cube library uses statically allocated buffers and a shallower call stack.

4.2.3.0.3 Execution time. STM32's USB initialisation is consistently 61.7 % faster across both HID and CDC. After initialisation, the two platforms perform within 1–2 % of each other, as the USB protocol itself (enumeration handshake, bulk/interrupt transfers) dictates the timing.

4.2.3.0.4 Design philosophy. STM32Cube separates concerns: HAL handles clocks and peripheral access, the middleware handles protocol logic, and the application owns configuration through user-editable files (`usbd_conf.c`, `usbd_desc.c`). This modular design gives developers explicit control and reusability at the cost of a larger total ROM. GigaDevice takes a more monolithic approach — smaller HAL, but a thicker middleware that bundles configuration internally, reducing ROM but also reducing the user's configuration flexibility.

4.2.3.0.5 Enhancement proposals. The data suggests two concrete areas where STM32Cube could reduce ROM without sacrificing functionality:

1. **HAL RCC optimisation.** The HAL RCC module accounts for ≈4.5 KB in every USB project regardless of the clocks actually configured. A conditionally compiled or link-time-optimised RCC

driver that includes only the clock paths used would significantly narrow the HAL gap.

2. **HAL/LL USB driver consolidation.** The combined HAL and LL USB driver (≈5 KB) provides a comprehensive abstraction but may include code paths (e.g. High-Speed, OTG, DMA) that Full-Speed-only projects never execute. Finer compile-time selection of USB features could reduce this cost.

4.3 FreeRTOS integration

FreeRTOS is the most widely adopted real-time operating system for Cortex-M microcontrollers [26]. Both STMicroelectronics and GigaDevice ship FreeRTOS integration packages as part of their firmware offerings. We benchmark the two implementations using FreeRTOS v11.2, running on the same two boards at 160 MHz with a 1 ms tick (time slice) per task.

4.3.1 Scenario design

We defined three scheduling scenarios that stress progressively more complex RTOS mechanisms:

1. **Basic processing** — a single counting thread runs at low priority alongside a high-priority reporting thread. The counting thread increments a global counter in a tight loop; the reporting thread wakes every 30 s and prints the counter delta. This isolates single-task throughput with minimal scheduler overhead.

2. **Cooperative processing** — five threads at the same low priority, each incrementing its own counter and then calling `taskYIELD()` to voluntarily yield the processor. A high-priority reporter prints all five counters every 30 s. This exercises voluntary context switching under equal-priority round-robin scheduling.

3. **Preemptive processing** — five threads at ascending priorities. Each thread resumes the next-higher-priority thread (which preempts it), increments its counter, then suspends itself. A high-priority reporter prints all five counters every 30 s. This exercises the full preemptive scheduler: suspend, resume, and priority-based preemption.

Table 4.9 summarises the three scenarios.

Table 4.9: FreeRTOS benchmark scenarios.

Scenario	What it exercises	Metric
Basic	Single-thread throughput, minimal scheduler overhead	Thread iterations/s
Cooperative	Voluntary context switching (`taskYIELD`)	Total thread iterations/30 s
Preemptive	Priority-based preemption, suspend/resume	Total thread iterations/30 s

4.3.2 Results

4.3.2.1 Basic processing

Table 4.10 shows the performance result and Table 4.11 the footprint.

Table 4.10: FreeRTOS basic processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/s	47 943	47 955	−0.025 %

Table 4.11: FreeRTOS basic processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	14 266	10 128	+40.9 %
RAM	11 960	11 716	+2.1 %

Performance is virtually identical (−0.025 %), confirming that the counting loop is CPU-bound and both FreeRTOS ports schedule it with the same efficiency. The ROM difference (+40.9 %) is significant; the RAM difference is negligible.

4.3.2.2 Cooperative processing

Table 4.12: FreeRTOS cooperative processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/30 s	1 220 066	1 200 666	+1.6 %

Table 4.13: FreeRTOS cooperative processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	15 422	11 224	+37 %
RAM	10 972	10 728	+2.3 %

STM32 completes 1.6 % more thread iterations over 30 seconds, indicating a marginally faster `taskYIELD()` context switch. The footprint gap narrows slightly to +37 % ROM.

4.3.2.3 Preemptive processing

Table 4.14: FreeRTOS preemptive processing — performance.

Metric	STM32U575	GD32E507	Difference
Thread iterations/30 s	294 615	285 144	+3.3 %

Table 4.15: FreeRTOS preemptive processing — footprint (bytes).

Metric	STM32U575	GD32E507	Difference
ROM	16 226	12 028	+35 %
RAM	10 996	10 748	+2.3 %

Preemptive scheduling is the most demanding scenario, involving suspend, resume, and priority-based task switching. STM32 achieves 3.3 % more iterations, the largest performance advantage across the three FreeRTOS scenarios. The ROM gap (+35 %) is consistent with the other scenarios.

4.3.2.4 Per-layer ROM breakdown

To understand *where* the ROM overhead originates, Table 4.16 breaks down the ROM footprint into three architectural layers — application, HAL drivers, and FreeRTOS middleware stack — for all three scenarios.

Table 4.16: FreeRTOS ROM breakdown by layer (bytes).

Scenario	Layer	STM32U575	GD32E507	Diff.
Basic	Application	4 531	4 278	+6 %
	HAL drivers	4 927	1 078	+357 %
	MW stack	4 808	4 772	+0.7 %
Cooperative	Application	5 823	5 484	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	4 688	4 662	+0.6 %
Preemptive	Application	6 191	5 810	+6 %
	HAL drivers	4 911	1 078	+355 %
	MW stack	5 124	5 140	−0.3 %

The pattern is strikingly consistent across all three scenarios:

- The **FreeRTOS middleware stack** itself is virtually identical in size (≤1 % difference). Both vendors ship the same upstream FreeRTOS kernel, so this is expected.
- The **application layer** is ≈6 % larger on STM32, reflecting slightly more initialisation code in the STM32Cube project template.
- The **HAL layer** is 355–357 % larger on STM32, entirely accounting for the total ROM gap. The dominant contributor is the **HAL RCC** module, which configures the clock tree.

4.3.3 Interpretation

4.3.3.0.1 Performance. FreeRTOS scheduling performance is nearly identical between the two platforms in the basic scenario (−0.025 %) and slightly favours STM32 in the cooperative (+1.6 %) and preemptive (+3.3 %) scenarios. The advantage grows with scheduler complexity, suggesting that the STM32Cube FreeRTOS port has a marginally more efficient context-switch implementation. However, these differences are small enough that application-level performance is unlikely to be affected.

4.3.3.0.2 ROM footprint. The ROM overhead follows the same pattern as the USB stack: a 35–41 % total increase driven almost entirely by the HAL layer, specifically the HAL RCC clock-

configuration module. Crucially, the FreeRTOS middleware itself contributes no measurable overhead — the gap is in the vendor's hardware-abstraction code, not in the RTOS kernel.

4.3.3.0.3 RAM footprint. RAM differences are negligible (2.1–2.3 %), indicating that both FreeRTOS ports allocate task stacks and kernel structures similarly.

4.3.3.0.4 Enhancement proposals. The FreeRTOS results reinforce the conclusion from the USB benchmark: the HAL RCC module is the single largest contributor to STM32's ROM overhead across *all* middleware stacks. A targeted effort to slim the RCC driver — through dead-code elimination, conditional compilation based on the clocks actually used, or link-time optimisation — would benefit every STM32 project, not just FreeRTOS.

Conclusion

The ST-versus-GD32 comparison across two middleware stacks (USB device and FreeRTOS) and five scenarios reveals a consistent pattern:

- **Performance:** STM32 matches or slightly exceeds GD32 in every scenario. USB initialisation is 61.7 % faster; FreeRTOS preemptive scheduling is 3.3 % faster; steady-state throughput is equivalent.
- **ROM footprint:** STM32 consumes 35–88 % more flash, with the HAL layer (principally HAL RCC and, for USB, HAL/LL USB) responsible for the bulk of the difference. The middleware stacks themselves are comparable in size.
- **RAM footprint:** STM32 uses significantly less RAM than GD32 in USB scenarios (57–65 % less) thanks to smaller default stack/heap allocations and more static memory management. FreeRTOS RAM is nearly identical.
- **Architectural trade-off:** STM32Cube's modular, layered design (separate HAL, middleware, and user-configuration files) provides greater flexibility and reusability but at a measurable ROM cost. GigaDevice's more monolithic approach yields smaller ROM by bundling driver and configuration logic into the middleware, at the expense of less user-visible modularity.

The single most impactful enhancement for STM32 would be **optimising the HAL RCC module**: it appears in every project and consistently accounts for the largest share of the HAL ROM gap. A

secondary improvement for USB projects would be **finer-grained compile-time selection** of USB driver features (Full-Speed vs High-Speed, device vs OTG) to exclude unused code paths.

Chapter 5

Chapter 5 title

Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

5.1 Section1

5.1.1 Subsection1

5.1.2 Subsection2

5.1.2.1 Subsubsection1

5.1.2.2 Subsubsection2

5.1.2.2.1 Paragraph a

5.1.2.2.2 Paragraph b

5.2 Section2

5.2.1 Subsection1

5.2.2 Subsection2

5.2.2.1 Subsubsection1

5.2.2.2 Subsubsection2

5.2.2.2.1 Paragraph a

5.2.2.2.2 Paragraph b

Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

General Conclusion and Perspectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent nec dapibus justo. Donec sagittis vulputate ante sed porttitor. Suspendisse sit amet nisl massa. Curabitur nec nisl condimentum, egestas ex vitae, dapibus enim. Etiam iaculis, erat faucibus pellentesque sagittis, nisi justo sollicitudin nibh, et condimentum augue massa non turpis. Proin commodo enim fermentum suscipit condimentum. Maecenas molestie, dui nec vestibulum rhoncus, arcu nisl faucibus neque, a ornare nisi massa ac eros. Aenean id velit sit amet lacus mattis varius. Donec fringilla massa sed nisi eleifend, a aliquet mi tempus. Nunc posuere euismod est, nec tristique augue lobortis non. Sed sodales sem ut metus tempus ullamcorper.

Nullam fermentum id mauris suscipit varius. Quisque tristique tempor fringilla. Nam porta tincidunt orci in consectetur. Suspendisse nec sem nisi. Suspendisse potenti. Sed sodales aliquam libero, at dapibus ligula accumsan non. Quisque congue et urna a consectetur. Nullam maximus venenatis ornare. Donec in luctus urna, vel rhoncus lectus. Etiam lobortis, lacus nec gravida iaculis, magna nulla iaculis leo, quis congue nunc tortor pellentesque lorem. Sed fermentum vulputate dui, ac malesuada eros molestie sit amet. Proin fringilla elit justo, in posuere urna porttitor et. Curabitur dictum justo vitae metus pellentesque, eget feugiat sem feugiat.

Curabitur in mauris eu felis cursus auctor eget ut massa. Curabitur eleifend consectetur magna in ultrices. Etiam ut semper turpis. Morbi sed ipsum nec sem rutrum blandit sit amet vitae felis. Vestibulum vitae hendrerit diam. Aliquam pellentesque est mi, et tempus nisl laoreet in. Maecenas consequat augue a ante congue dignissim. In condimentum erat in volutpat tempus. Mauris vulputate, enim ut dignissim varius, lorem purus tristique sapien, lobortis accumsan dolor lorem quis eros. Duis vel imperdiet metus, non suscipit tortor.

Appendix A

Glossary

Telnet: Telnet is a network protocol that allows users to establish a remote terminal connection to a computer or network device over a network.

SSH (Secure Shell): SSH is a network protocol used for secure remote administration and secure file transfers.

SNMP (Simple Network Management Protocol): SNMP is a network protocol designed for managing and monitoring network devices and systems.

Appendix B

Acronyms

IP	Internet Protocol
CNS	Cloud and Network Services
MN	Mobile Networks
NI	Network Infrastructure
IoT	Internet of Things
NFV	Network Functions Virtualization
SDN	Software-Defined Network
TCP	Transmission Control Protocol
7750 SR	Nokia 7750 Service Router
OSPF	Open short Path First
BGP	Border Gateway Protocol
VPN	Virtual Private Network
MPLS	Multiprotocol Label Switching
EVE-NG	Emulated Virtual Environment - Next Generation
VM	Virtual Machine
SSH	Secure Shell
WinSCP	Windows Secure Copy
FTP	File Transfer Protocol
SSL	Secure Sockets Layer
TLS	Transport Layer Security
FTPS	FTP over SSL/TLS

SFTP	Secure File Transfer Protocol
SCP	Secure Copy Protocol
CLI	Command-line interface
MD-CLI	Model-Driven Command Line
API	Application Programming Interface
re	Regular Expression
IGP	Internal Gateway Protocol
IS-IS	Intermediate-System Intermediate-System
SPF	Shortest Path First
SF	Switch Fabric
CPM	Control Processing Module
IOM	Input/Output Modules
MDA	Media Dependent Adapters
QoS	Quality of Service
IPsec	Internet Protocol Security
VSR	Virtual Services Router
DDoS	Distributed Denial-of-Service
SROS	Service Router Operating System
NETCONF	Network Configuration Protocol
YANG	Yet Another Next Generation
MP-BGP	Multiprotocol Border Gateway Protocol
IPv6	Internet Protocol Version 6
AS	Autonomous Systems
iBGP	Internal Border Gateway Protocol
eBGP	External Border Gateway Protocol
LAG	Link Aggregation Group
VPRN	Virtual Private Routed Network
SNMP	Simple Network Management Protocol
Telnet	Telecommunication Network
YAML	Yet Another Markup Language
XML	eXtensible Markup Language

MCU	Microcontroller Unit
RAM	Random-Access Memory
HAL	Hardware Abstraction Layer
SDK	Software Development Kit
GPIO	General-Purpose Input/Output
USB	Universal Serial Bus
HID	Human Interface Device
GUI	Graphical User Interface
IAR	IAR Systems (toolchain vendor)
EWARM	Embedded Workbench for ARM
LLM	Large Language Model
AI	Artificial Intelligence
OAuth	Open Authorization
JSON	JavaScript Object Notation
CSV	Comma-Separated Values
XLSX	Office Open XML Spreadsheet
PDF	Portable Document Format
SQL	Structured Query Language
SQLite	Self-contained SQL Database Engine

Appendix C

Some subject you want to expand on

You can add more appendices depending on the subjects.

```
[root@host ~]# You can change the language and caption.
```

Listing C.1: Bash example

```
# Solve the quadratic equation ax**2 + bx + c = 0

# import complex math module
import cmath

a = 1
b = 5
c = 6

# calculate the discriminant
d = (b**2) - (4*a*c)

# find two solutions
sol1 = (-b-cmath.sqrt(d))/(2*a)
sol2 = (-b+cmath.sqrt(d))/(2*a)

print('The solution are {0} and {1}'.format(sol1,sol2))
```

```
20
```

Listing C.2: Python example

Column1	Column2
Element11	Element21
Element12	Element22
Element13	Element23

Table C.1: Table Example

Table C.2: Long table Example

Cell	Description
Element11	Element21
Element12	Element22
Element13	Element23
Element14	Element24
Element15	Element25
Element16	Element26
Element17	Element27
Element18	Element28
Element19	Element29
Element110	Element210
Element111	Element211
Element112	Element212
Element113	Element213
Element114	Element214

Bibliography

[1] STMicroelectronics, *About ST*. [Online]. Available: https://www.st.com/content/st_com/en/about/st_company_information.html. Accessed: 2026.

[2] STMicroelectronics, *2024 Annual Report and Financial Highlights*. [Online]. Available: https://investors.st.com/. Accessed: 2026.

[3] STMicroelectronics, *Strategy and Innovation*. [Online]. Available: https://www.st.com/content/st_com/en/about/innovation---technology/innovation---technology.html. Accessed: 2026.

[4] STMicroelectronics, *STM32 32-bit Arm Cortex MCUs*. [Online]. Available: https://www.st.com/en/microcontrollers-microprocessors/stm32-32-bit-arm-cortex-mcus.html. Accessed: 2026.

[5] Arm Ltd., *Cortex-M Series Processors*. [Online]. Available: https://developer.arm.com/Processors/Cortex-M. Accessed: 2026.

[6] STMicroelectronics, *STM32Cube Ecosystem*. [Online]. Available: https://www.st.com/en/ecosystems/stm32cube.html. Accessed: 2026.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[8] Python Software Foundation, *The Python Language Reference, version 3.11*. [Online]. Available: https://docs.python.org/3.11/. Accessed: 2026.

[9] The Qt Company, *Qt for Python (PySide6) Documentation*. [Online]. Available: https://doc.qt.io/qtforpython/. Accessed: 2026.

[10] The pandas development team, *pandas: Powerful Python Data Analysis Toolkit*. [Online]. Available: https://pandas.pydata.org/. Accessed: 2026.

[11] E. Gazoni and C. Clark, *openpyxl: A Python Library to Read and Write Excel 2010 xlsx/xlsm Files*. [Online]. Available: https://openpyxl.readthedocs.io/. Accessed: 2026.

[12] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. [Online]. Available: https://matplotlib.org/.

[13] SQLite Development Team, *SQLite Documentation*. [Online]. Available: https://www.sqlite.org/docs.html. Accessed: 2026.

[14] PyInstaller Development Team, *PyInstaller Manual*. [Online]. Available: https://pyinstaller.org/. Accessed: 2026.

[15] IAR Systems, *IAR Embedded Workbench for Arm — C/C++ Development Guide (ILINK Linker and Linker Map File)*. [Online]. Available: https://www.iar.com/embedded-development-tools/iar-embedded-workbench. Accessed: 2026.

[16] GitHub, *GitHub Copilot Documentation*. [Online]. Available: https://docs.github.com/en/copilot. Accessed: 2026.

[17] Anthropic, *Claude API Documentation*. [Online]. Available: https://docs.anthropic.com/. Accessed: 2026.

[18] OpenAI, *OpenAI API Reference*. [Online]. Available: https://platform.openai.com/docs/. Accessed: 2026.

[19] Ollama, *Ollama: Run Large Language Models Locally*. [Online]. Available: https://ollama.com/. Accessed: 2026.

[20] GitHub, *GitHub Models Documentation*. [Online]. Available: https://docs.github.com/en/github-models. Accessed: 2026.

[21] NXP Semiconductors, *MCUXpresso SDK Documentation*. [Online]. Available: https://mcuxpresso.nxp.com/. Accessed: 2026.

[22] Microsoft, *Visual Studio Code Documentation*. [Online]. Available: https://code.visualstudio.com/docs. Accessed: 2026.

[23] Microsoft, *GitHub Copilot Agent Mode in VS Code*. [Online]. Available: https://code.visualstudio.com/docs/copilot/chat/chat-agent-mode. Accessed: 2026.

[24] GigaDevice Semiconductor, *GD32 Microcontrollers*. [Online]. Available: https://www.gigadevice.com/microcontroller. Accessed: 2026.

[25] STMicroelectronics, *STM32U575/585 Datasheet — Ultra-low-power with FPU Arm Cortex-M33 MCU*. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32u575zi.pdf. Accessed: 2026.

[26] Amazon Web Services, *FreeRTOS — Real-Time Operating System for Microcontrollers*. [Online]. Available: https://www.freertos.org/. Accessed: 2026.